NEXT GEN
SOFTWARE HUB

CYBER SECURITY INTERNSHIP

WEEK 03
TASK 03 REPORT

Linux for Cybersecurity

Building Essential Linux Skills for
a Safer Digital World

TOPICS

- Basic Linux commands
- Permissions
- Scripting

TOOLS

- Kali Linux / Parrot OS

RESOURCES

- Linux Journey
- OverTheWire - Bandit

TASK

Complete Level 0–10
on Bandit wargame.

Week 3: Linux for Cybersecurity

PLANS :

Phase 1: Foundations (Weeks 1–4)



PREPARED BY
Selo David Sethibo
Cyber Security Intern

2026

1. INTRODUCTION

Linux is an important operating system in cybersecurity because it provides powerful tools and command-line utilities used for system administration, networking, security testing, digital forensics, penetration testing, and security analysis.

Unlike graphical interfaces where many operations can be performed through menus and windows, Linux provides a command-line environment where users can interact directly with files, directories, processes, permissions, and system resources.

Understanding Linux commands is therefore an important foundation for cybersecurity professionals. Security analysts and penetration testers frequently work with Linux systems and use commands to navigate directories, examine files, search for information, identify file types, manipulate data, and interact with remote systems.

This task focused on developing practical Linux skills through the OverTheWire Bandit wargame.

The Bandit wargame provides a series of progressively challenging Linux-based tasks. Each level requires the user to investigate the environment, identify relevant files or information, use appropriate Linux commands, and obtain the password required to progress to the next level.

For this practical, Bandit Levels 0-10 were completed.

The practical involved connecting to the Bandit server through SSH and using Linux commands such as ls, cat, cd, file, find, grep, sort, uniq, strings, and base64.

The practical also introduced important command-line concepts including hidden files, special filenames, wildcards, file properties, recursive searching, permission errors, output redirection, text searching, pipelines, duplicate data, readable strings, and Base64 encoding.

2. OBJECTIVES

The objectives of this task were:

- To understand the Linux operating system and command-line environment.
- To understand basic Linux commands.
- To learn how to navigate directories using the command line.
- To learn how to list files and directories.
- To understand how to read files from the command line.
- To understand hidden files in Linux.
- To understand how Linux handles special filenames.
- To understand file types.
- To understand wildcard characters.
- To understand file properties.
- To learn how to search for files using find.
- To understand Linux file permissions and permission errors.
- To understand output redirection.
- To learn how to search file contents using grep.
- To understand sorting and identifying unique data.
- To understand the strings command.
- To understand Base64 encoding and decoding.
- To practice connecting to a remote Linux system using SSH.
- To complete OverTheWire Bandit Levels 0–10.
- To develop practical Linux skills relevant to cybersecurity.

3. LINUX IN CYBERSECURITY

Linux is widely used in cybersecurity because it provides extensive command-line tools and utilities that allow security professionals to interact directly with systems.

Linux can be used for activities such as:

- Network security
- Penetration testing
- Vulnerability assessment
- Digital forensics
- Incident response
- System administration
- Security monitoring
- Malware analysis
- Network analysis
- Security automation

Security distributions such as Kali Linux and Parrot OS provide many security tools, but understanding the underlying Linux command line is important because cybersecurity tools frequently require users to navigate directories, manipulate files, search information, and execute commands.

The Bandit practical provided an opportunity to develop these basic skills in a controlled environment.

4. LINUX COMMAND-LINE ENVIRONMENT

The Linux command line allows users to interact with an operating system by entering commands into a terminal.

Instead of relying entirely on a graphical user interface, users can perform operations directly using commands.

Some of the commands used during this practical included:

Command	Purpose
Ls	Lists files and directories
cd	Changes directory
cat	Displays file contents
file	Identifies the type of a file
find	Searches for files and directories
grep	Searches text for patterns
Sort	Sorts lines of text
Uniq	Filters repeated lines
Strings	Extracts readable strings
Base64	Encodes or decodes Base64 data
Man	Displays command documentation

These commands were applied progressively as the Bandit levels became more difficult.

5. PRACTICAL ENVIRONMENT

The practical was performed using the OverTheWire Bandit wargame.

The environment consisted of a remote Linux server accessed through SSH.

Environment

Platform: OverTheWire Bandit

Protocol: SSH

SSH Port: 2220

Analysis/Practice Machine: Kali Linux

The Bandit server provided separate user accounts for the different levels.

The practical started with the **bandit0** account and progressed through the required levels.

Each level required obtaining the password needed to authenticate to the next level.

6. PRACTICAL METHODOLOGY

The practical followed a progressive process:

1. Connected to the Bandit server using SSH.
2. Logged in using the account provided for the current level.
3. Examined the available files and directories.
4. Identified the information required by the level.
5. Selected an appropriate Linux command or combination of commands.
6. Executed the command in the terminal.
7. Examined the resulting output.
8. Identified the password for the next level.
9. Used the password to authenticate to the next Bandit account.
10. Repeated the process until Level 10 was completed.

The practical therefore focused not only on executing commands but also on understanding why a particular command was appropriate for a particular problem.

7. BANDIT LEVEL 0: CONNECTING TO THE SERVER

The first level introduced the process of connecting to a remote Linux system using SSH.

SSH, or Secure Shell, provides a way to securely access a remote system through a command-line interface.

The Bandit server used a non-standard SSH port, 2220.

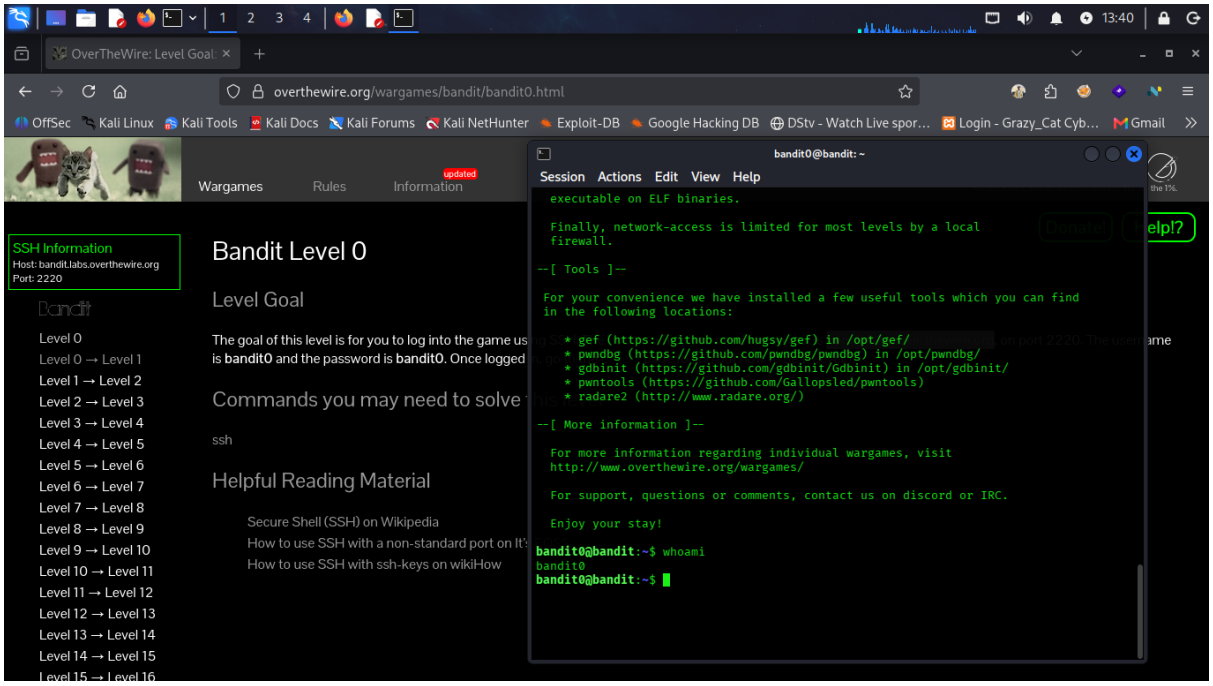
After successfully connecting, the Bandit environment was available through the terminal.

What was learned

This level introduced:

- SSH
- Remote Linux access

- User authentication
- SSH ports
- Linux terminal interaction



What the screenshot proves: It provides evidence that the remote Linux environment was successfully accessed using SSH.

8. BANDIT LEVEL 0 to LEVEL 1: READING THE README FILE

After logging into the first Bandit account, the contents of the current directory were examined.

The **ls** command was used to determine which files were available.

A file named: **readme** was present.

The **cat** command was then used to display the contents of the file.

The contents provided the password required to continue to the next level.

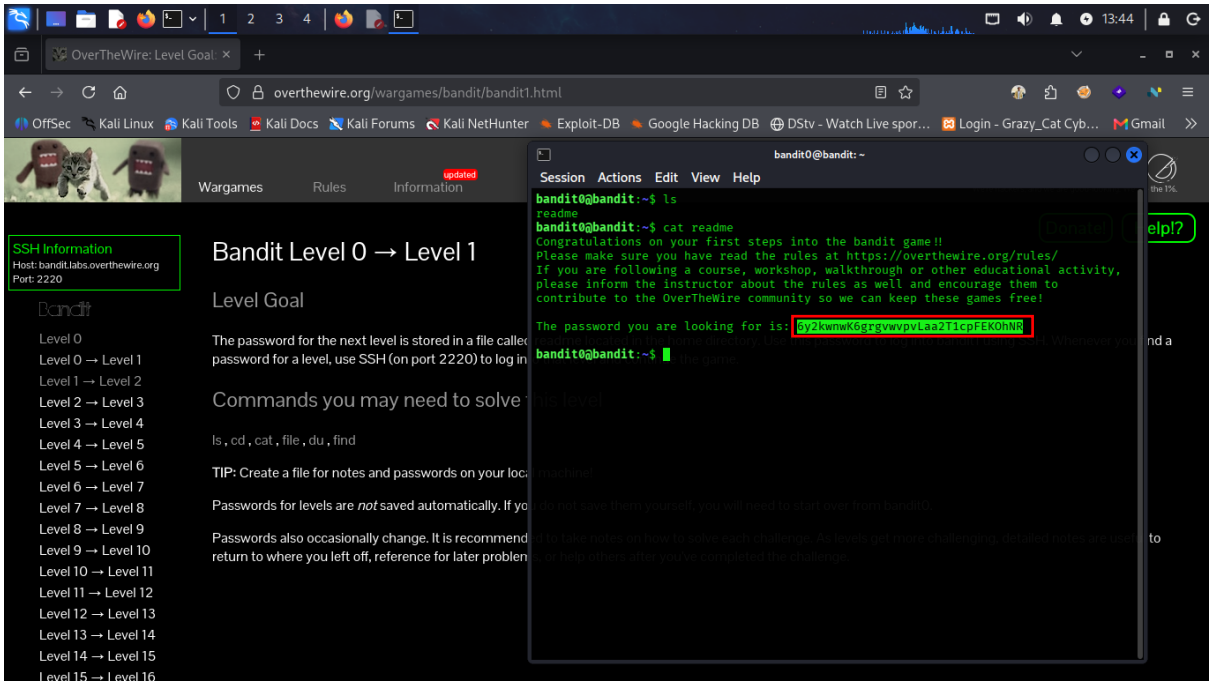
Commands used

- ls
- cat readme

What was learned

This level demonstrated two fundamental Linux commands:

- **ls** is used to list files and directories.
- **cat** is used to display the contents of a file.



What the screenshot proves: It demonstrates that the file was successfully identified and read using Linux commands.

9. BANDIT LEVEL 1 to LEVEL 2: HANDLING A SPECIAL FILENAME

The next level involved a file with a filename consisting of a single hyphen: **-**

A filename beginning with (**-**) can create a problem because many Linux commands interpret values beginning with a hyphen as command options.

For example, instead of interpreting (**-**) as a filename, a command may interpret it as an instruction or option.

The file was therefore accessed by explicitly specifying its path.

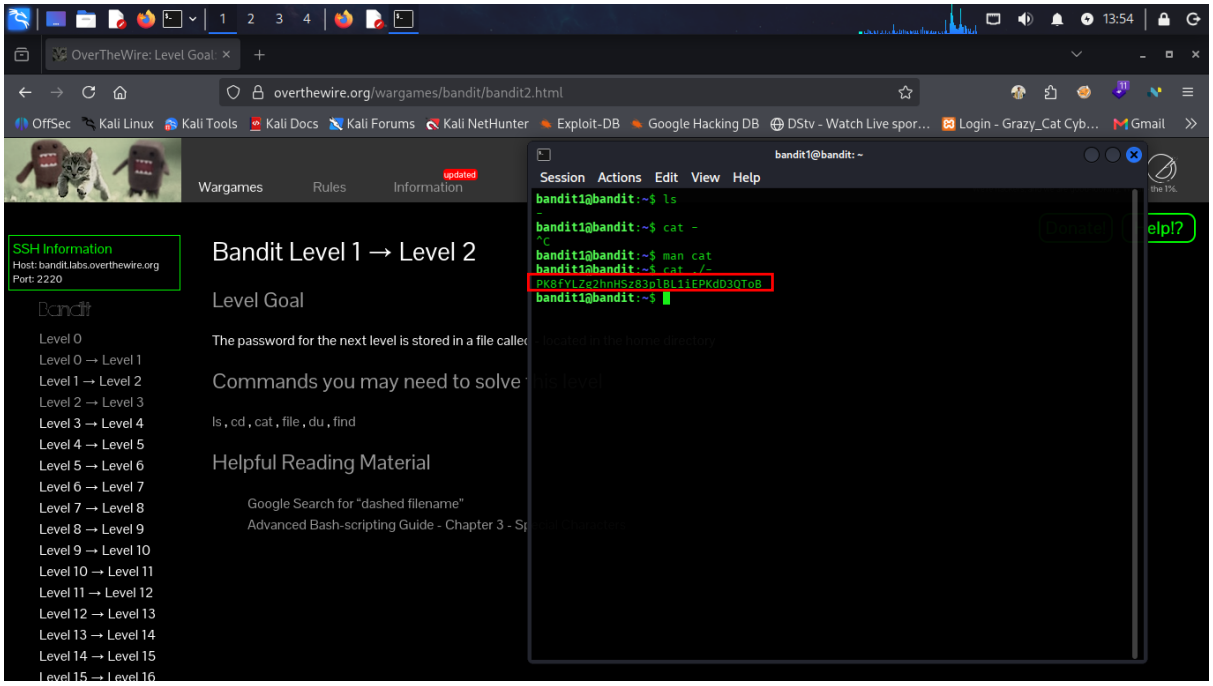
The approach used was: **cat ./-**

The (**./**) tells Linux that the object is located in the current directory.

What was learned

This level demonstrated:

- Relative paths
- Special filenames
- Command options
- The importance of **./**
- Safe handling of filenames beginning with (**-**)



What the screenshot proves: It demonstrates that the special filename was correctly interpreted as a file rather than as a command-line option.

10. BANDIT LEVEL 2 to LEVEL 3: FILENAMES CONTAINING SPACES

The next challenge involved a filename containing spaces: **--spaces in this filename--**

Spaces normally separate arguments in a Linux command.

Therefore, entering the filename without handling the spaces correctly could cause Linux to interpret the filename as multiple separate arguments.

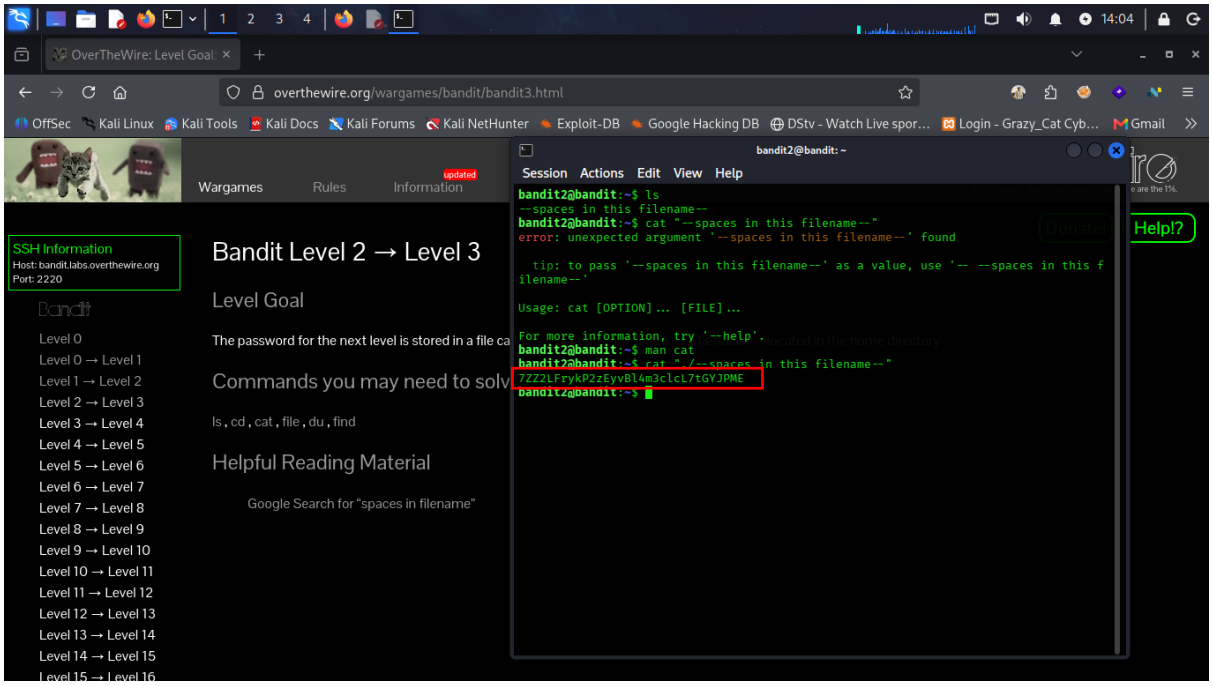
The filename was therefore handled using quotation marks and the current-directory path.

The approach was: **cat "./--spaces in this filename--"**

What was learned

This level demonstrated:

- Filenames containing spaces
- Quoting filenames
- Relative paths
- Command-line argument handling



What the screenshot proves: It demonstrates that the filename was correctly passed to the cat command as a single argument.

11. BANDIT LEVEL 3 to LEVEL 4: FINDING HIDDEN FILES

The next level introduced hidden files.

Linux hides files whose names begin with a period (.) when a normal ls command is used.

The directory was entered and its contents were examined.

The **ls -a** command was used to display hidden files as well.

The hidden file was then identified and examined.

The **file** command was useful for identifying the type of the discovered file before reading it.

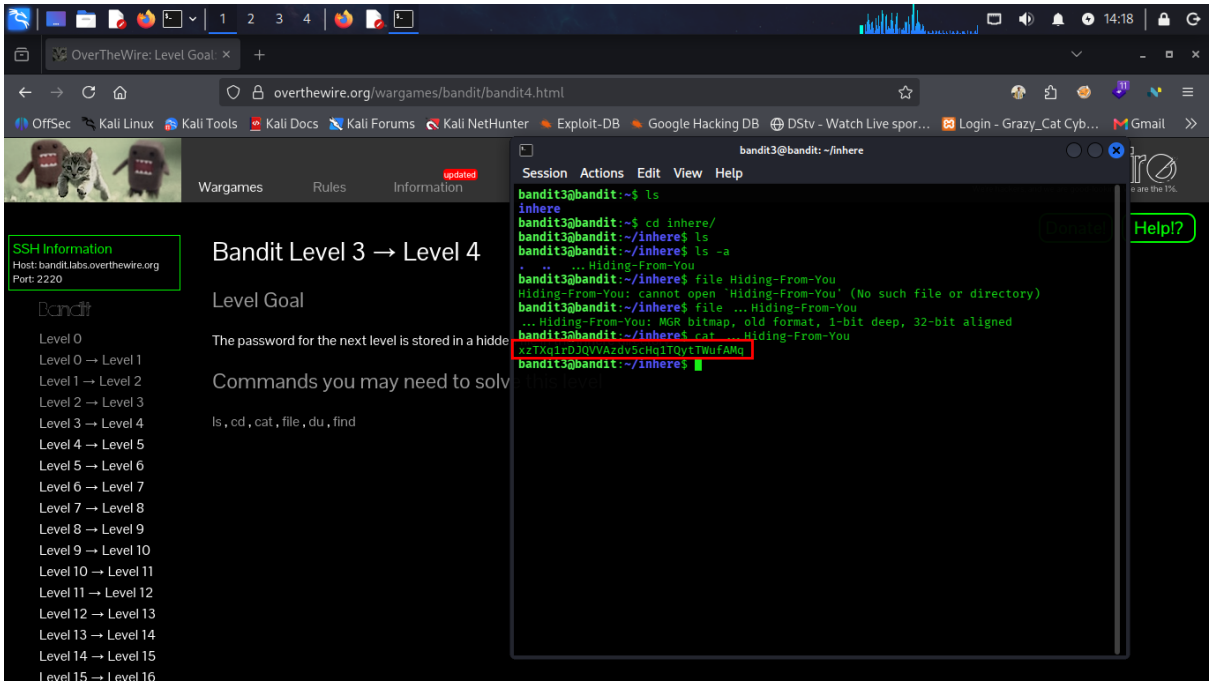
Commands used

- cd
- ls -a
- file
- cat

What was learned

This level demonstrated:

- Hidden files
- The -a option of ls
- Directory navigation
- File identification
- Reading hidden files



What the screenshot proves: It demonstrates that hidden Linux files can be discovered using appropriate command-line options.

12. BANDIT LEVEL 4 to LEVEL 5: IDENTIFYING THE HUMAN-READABLE FILE

This level involved multiple files inside a directory.

The objective was to identify the file containing human-readable text.

The **file** command was used to examine the files and determine their file types.

Wildcards were also used to examine multiple files efficiently.

For example: **file ./***

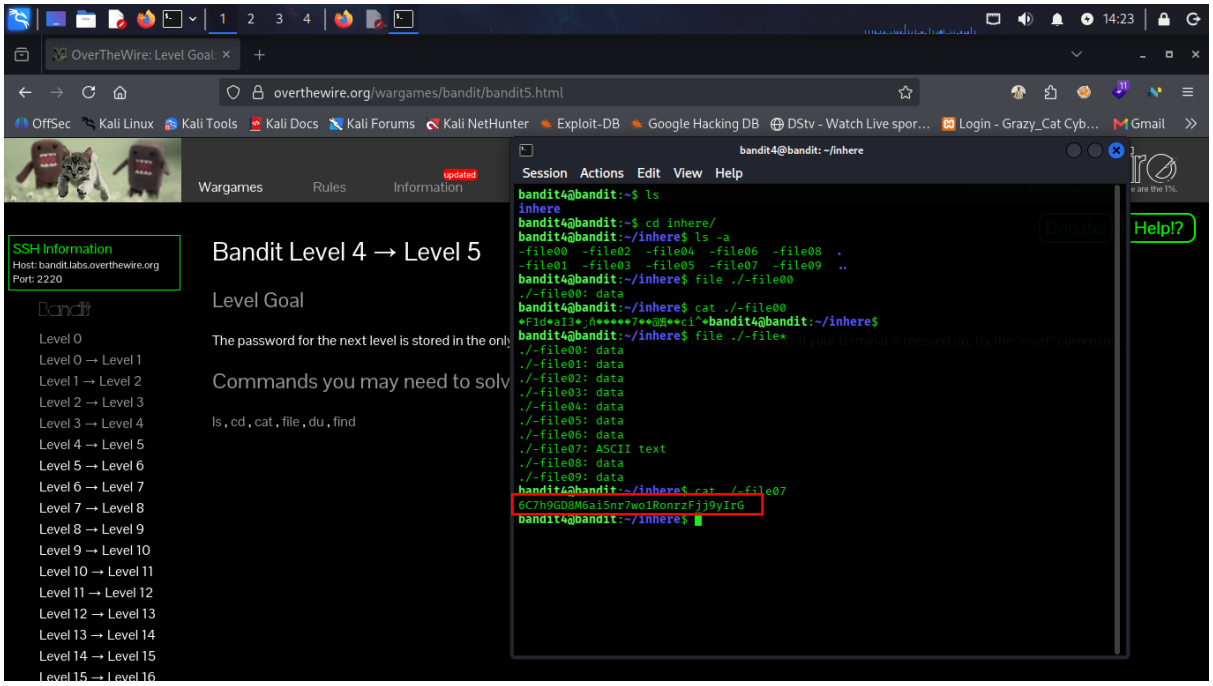
The files were then inspected to identify the relevant human-readable file.

What was learned

This level demonstrated:

- File identification
- Wildcards
- Batch examination of files
- Human-readable versus non-human-readable content

The wildcard * allows a command to operate on multiple matching files rather than requiring each filename to be entered individually.



What the screenshot proves: It demonstrates the use of file-type analysis to locate the correct file among multiple candidates.

13. BANDIT LEVEL 5 to LEVEL 6: SEARCHING BY FILE PROPERTIES

The next level required finding a file based on specific properties rather than simply knowing its filename.

The **find** command was used to search for the required file.

The relevant property included the file size.

The notes recorded the use of: **find -size 1033c**

The **c** represents bytes when used with the **-size** option.

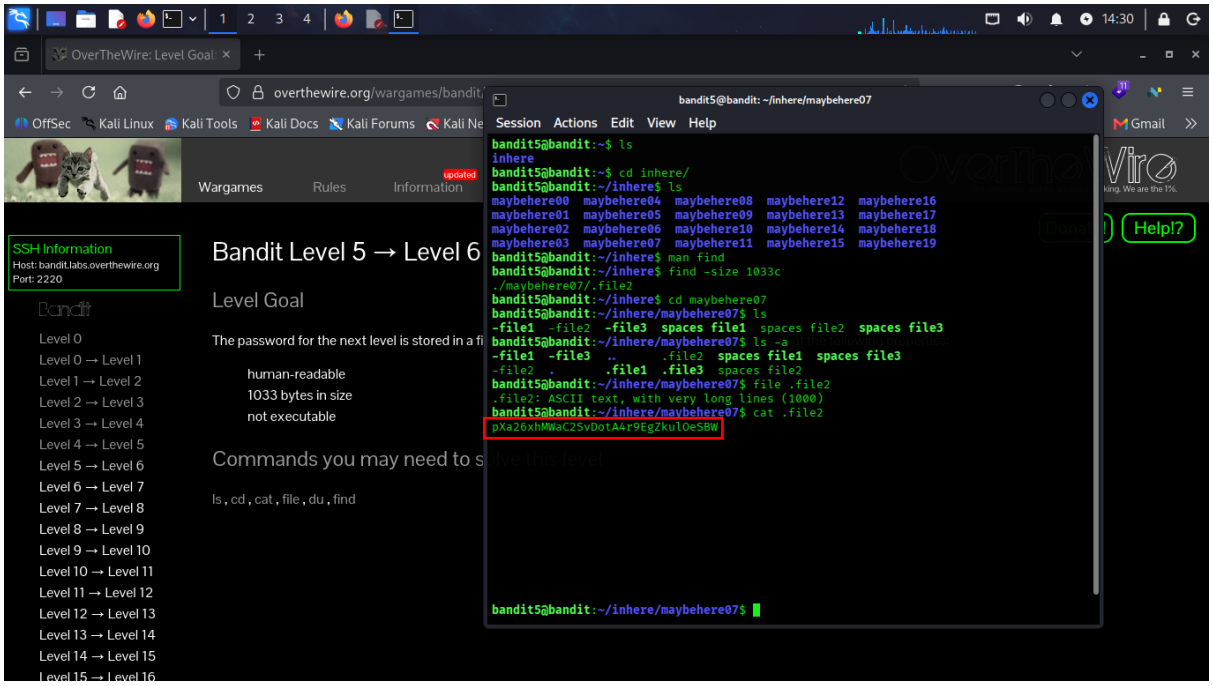
The correct file was then located within the directory structure and its contents were read.

What was learned

This level demonstrated:

- The find command
- File size searching
- Recursive directory searching
- File properties
- Using man find to understand command options

The **man** command was also used to consult Linux documentation.



What the screenshot proves: It demonstrates that Linux can search for files according to specific properties rather than requiring the exact filename to be known beforehand.

14. BANDIT LEVEL 6 to LEVEL 7: SYSTEM-WIDE FILE SEARCH

This level increased the difficulty by requiring a search across the filesystem.

The required file had specific ownership and size characteristics.

The search involved:

- User ownership
- Group ownership
- File size

The command recorded in the notes was: **find / -user bandit7 -group bandit6 -size 33c 2>/dev/null**

The search began from /, meaning the entire filesystem was considered.

The command also used: **2>/dev/null**

This redirects error output to /dev/null.

This is particularly useful when searching the filesystem because some directories may not be accessible to the current user.

The correct file was then read to obtain the password.

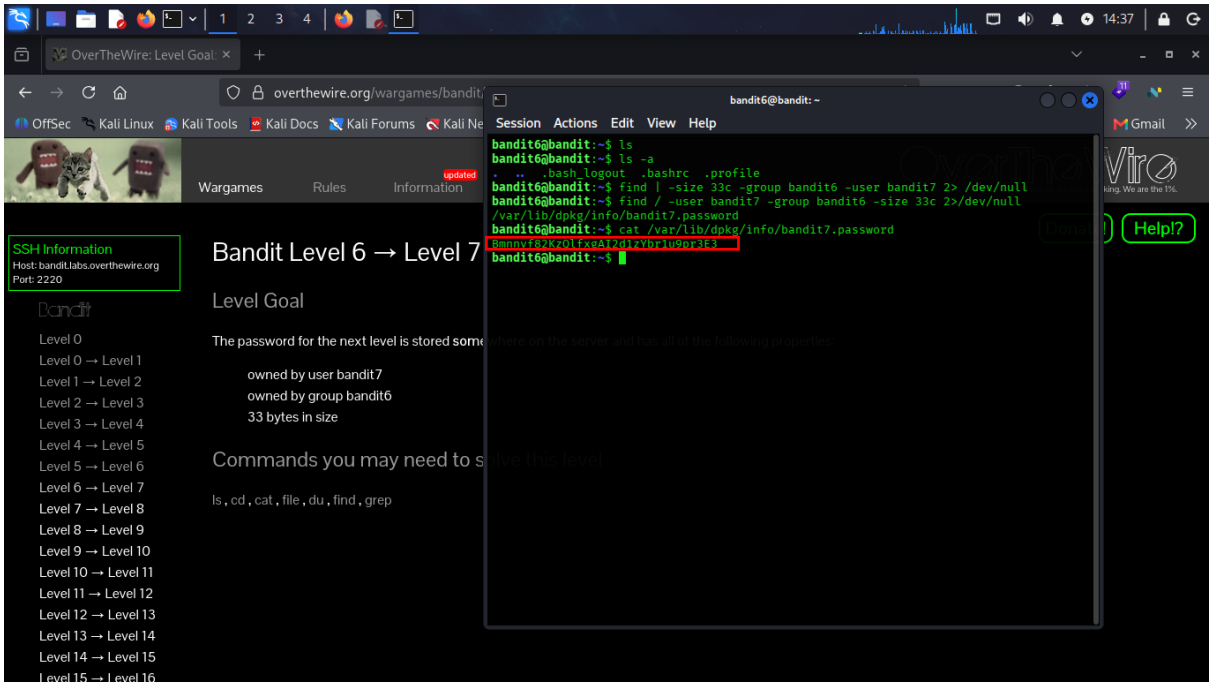
What was learned

This level demonstrated several important Linux concepts:

- Searching the entire filesystem
- File ownership

- Group ownership
- File size
- Error redirection
- /dev/null
- Permission errors

The 2>/dev/null technique is particularly useful in cybersecurity because system-wide searches can generate many permission-related errors.



What the screenshot proves: It demonstrates the ability to locate files using multiple attributes and handle permission errors during a system-wide search.

15. BANDIT LEVEL 7 to LEVEL 8: SEARCHING FILE CONTENTS

This level required finding a specific piece of information inside a large file.

The file was: **data.txt**

The keyword that needed to be located was: **millionth**

The **grep** command was used to search the file for that specific text.

The command used was: **grep "millionth" data.txt**

grep is a powerful Linux utility for searching text within files.

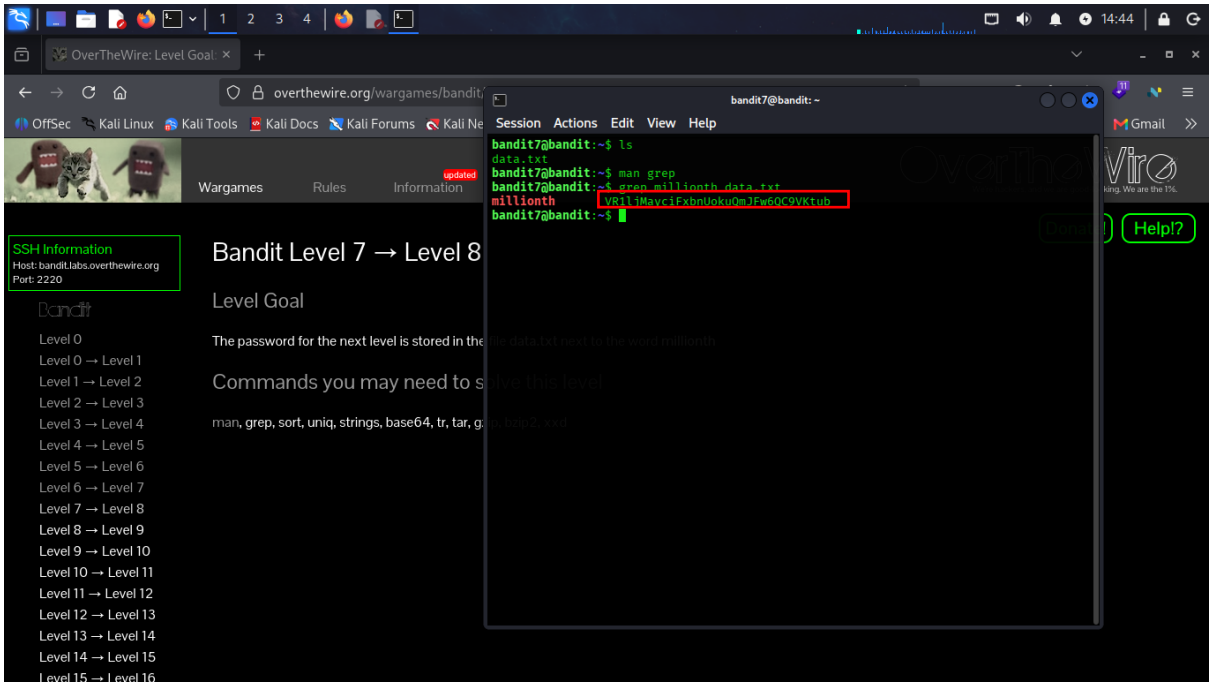
What was learned

This level demonstrated:

- Searching file contents
- Pattern matching
- grep

- Filtering large amounts of text

Instead of manually reading the entire file, grep allows the user to locate the relevant line directly.



What the screenshot proves: It demonstrates the ability to search large text files efficiently using a specific keyword.

16. BANDIT LEVEL 8 to LEVEL 9: FINDING THE UNIQUE LINE

This level involved a file containing many lines of data.

The objective was to identify the line that occurred only once.

The solution involved combining two commands: **sort data.txt | uniq -u**

The sort command arranges the lines in order.

The output is then passed to uniq using a pipe (|).

The -u option identifies unique lines.

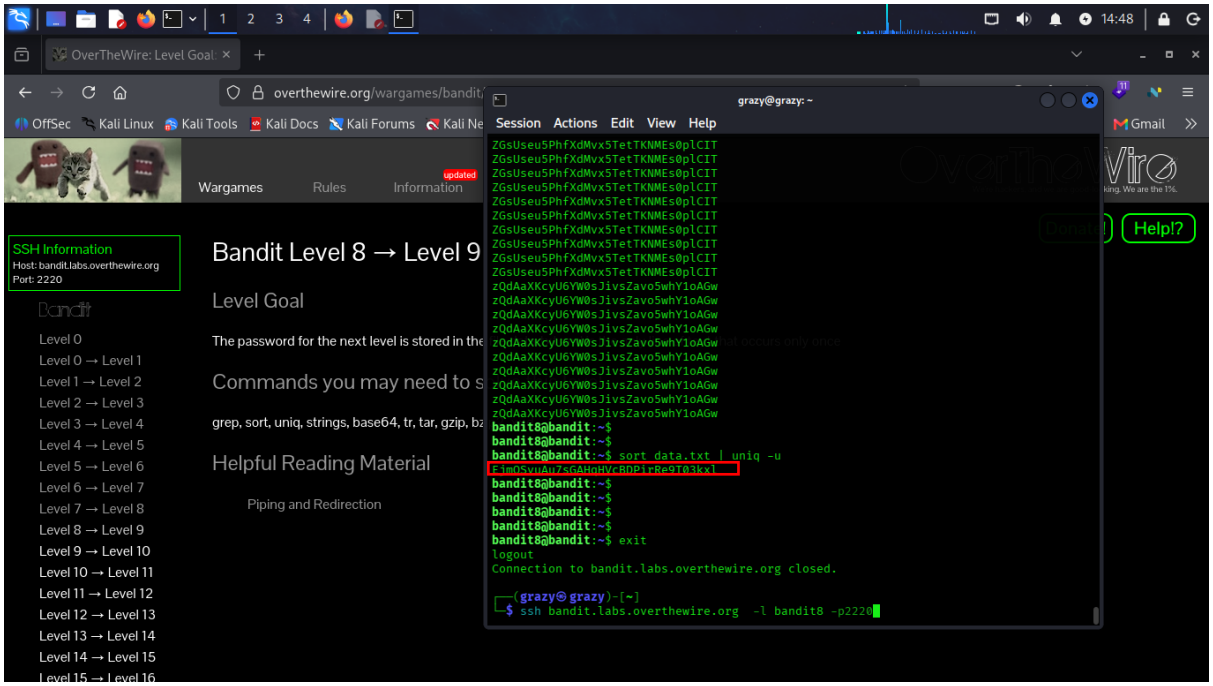
The pipe therefore connects the output of one command to the input of another.

What was learned

This level demonstrated:

- Sorting
- Unique values
- Command pipelines
- Combining Linux commands
- Data processing through the command line

The important concept was not simply knowing the commands individually, but understanding how commands can be combined to solve a problem.



What the screenshot proves: It demonstrates the use of command pipelines to process and filter information efficiently.

17. BANDIT LEVEL 9 to LEVEL 10: EXTRACTING READABLE STRINGS

The next level involved data that contained readable information mixed with non-readable content.

The **strings** command was used to extract sequences of printable characters.

The output was then filtered using grep.

The command used was: **strings data.txt | grep "==="**

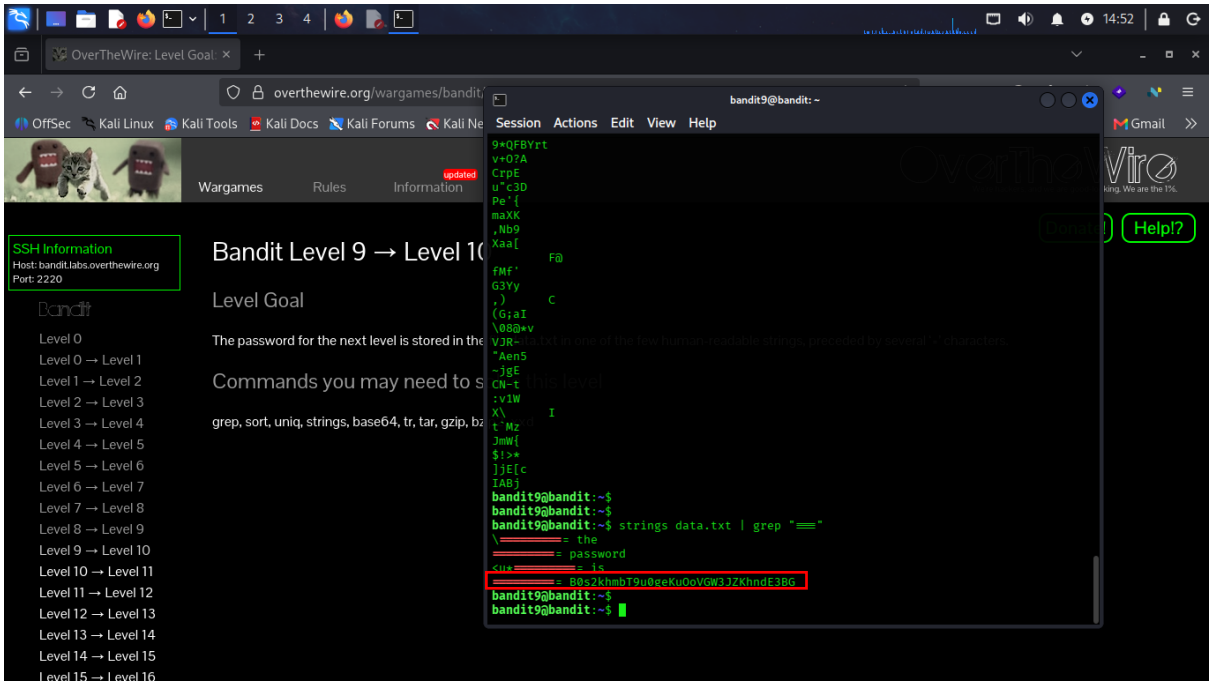
The strings command extracts readable text from a file, while grep searches the resulting output for the required pattern.

What was learned

This level demonstrated:

- Extracting readable strings
- Searching command output
- Command pipelines
- Filtering data
- Working with files that are not simply plain text

This type of technique can also be useful in cybersecurity when examining files for readable information.



What the screenshot proves: It demonstrates how readable information can be extracted from a file and filtered to locate relevant data.

18. BANDIT LEVEL 10: BASE64 DECODING

The final completed level in this task introduced Base64 encoding.

The file contained encoded data rather than plain readable text.

The base64 utility was used with the decode option.

The command used was: **base64 -d data.txt**

The -d option tells the command to decode the Base64 data.

The resulting output revealed the password required for the next level.

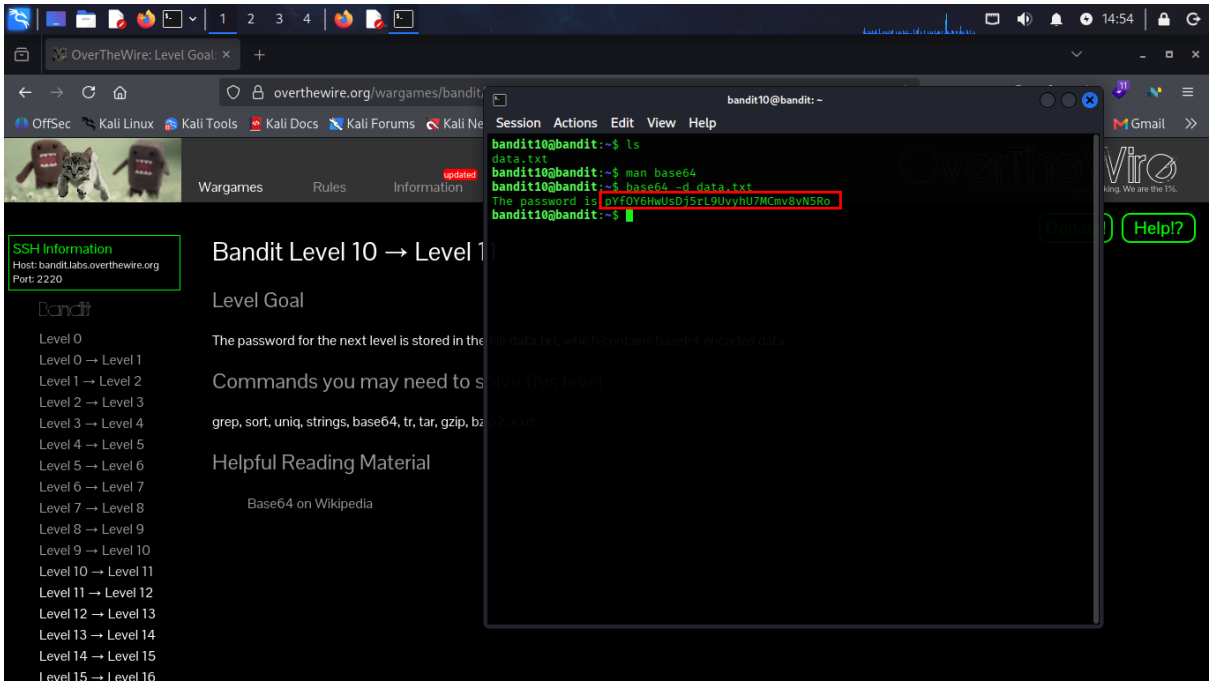
What was learned

This level demonstrated:

- Base64 encoding
- Base64 decoding
- Command-line data transformation
- Recognizing encoded information

Base64 is an encoding mechanism rather than encryption. It changes the representation of data but does not provide confidentiality.

This distinction is important in cybersecurity because encoded data should not automatically be considered secure or encrypted.



What the screenshot proves: It demonstrates that the encoded data was successfully decoded using the Linux command line.

19. COMMANDS AND LINUX CONCEPTS PRACTICED

Throughout Levels 0-10, several Linux commands and concepts were introduced.

Command / Concept	What was learned
ssh	Remote Linux access
ls	Listing files
cd	Navigating directories
cat	Reading files
./	Referencing files in the current directory
ls –a	Displaying hidden files
file	Identifying file types
*	Wildcard matching
find	Searching for files
-size	Searching by file size
-user	Searching by owner
-group	Searching by group
2>/dev/null	Suppressing error output
grep	Searching text
sort	Sorting data
uniq -u	Finding unique lines
|	Piping command output
strings	Extracting readable strings
base64 -d	Decoding Base64 data
man	Reading command documentation

20. IMPORTANCE OF LINUX COMMANDS IN CYBERSECURITY

The practical demonstrated that Linux commands are not only useful for ordinary system administration but can also support cybersecurity activities.

For example, the find command can be used during investigations to locate files with particular properties.

The grep command can be used to search logs or configuration files for specific information.

The strings command can help identify readable information contained within files.

File-type identification can help determine whether a file is text, binary, executable, or another type.

Command pipelines allow multiple operations to be combined, making it possible to process large amounts of information efficiently.

These skills can therefore support activities such as:

- Security investigations
- Log analysis
- File analysis
- Incident response
- System administration
- Penetration testing
- Digital forensics
- Vulnerability assessment

21. SECURITY RELEVANCE OF THE BANDIT PRACTICAL

Although the Bandit wargame is designed as a learning environment, the skills developed through the levels are relevant to real cybersecurity work.

The practical required identifying information from a Linux environment without relying on a graphical interface.

For example, finding hidden files can be useful when investigating a system for unusual or suspicious files.

Searching files by ownership, group, or size can help security analysts locate files that meet particular investigation criteria.

Searching text using grep can help identify specific events or indicators in large log files.

Using strings can help analysts identify readable content within files.

Understanding Base64 is useful when examining data that has been encoded for transmission or storage.

The practical therefore developed the type of command-line problem-solving skills that are useful when working with Linux-based security environments.

22. CHALLENGES AND OBSERVATIONS

- One of the main observations from the practical was that Linux commands can often be combined to solve problems more efficiently.

For example: **sort data.txt | uniq -u**

combines two commands using a pipeline rather than requiring the data to be processed manually.

- Another important observation was the importance of understanding how Linux interprets filenames.
  - The challenges involving (**-**) and spaces demonstrated that filenames are not always straightforward command-line arguments.
  - The hidden-file challenge also demonstrated that information may not be visible when using basic commands. Additional command options may be required to reveal it.

The system-wide search demonstrated that permissions can affect command output. Using:

2>/dev/null

allowed the search to continue without displaying large numbers of permission-denied messages.

The practical therefore required not only memorizing commands but understanding the Linux environment and selecting the appropriate approach for each problem.

23. PRACTICAL RESULTS SUMMARY

Level	Main Concept	Main Tool/Command	Result
0	SSH connection	ssh	Successfully accessed Bandit
0-1	Reading a file	ls,cat	Password obtained
1-2	Special filename	cat ./-	Password obtained
2-3	Filename with spaces	cat "./..."	Password obtained
3-4	Hidden files	ls -a, file, cat	Password obtained
4-5	File identification	file, wildcard	Password obtained
5-6	File properties	find	Password obtained
6-7	System wide search	find, 2>/dev/null	Password obtained
7-8	Text searching	grep	Password obtained
8-9	Unique data	sort, uniq	Password obtained
9-10	Readable strings	strings, grep	Password obtained
10-11	Base64 decoding	Base64 -d	Password obtained

24. EVIDENCE OF COMPLETION

The practical was supported by terminal screenshots showing the successful completion of the Bandit levels.

The screenshots provide evidence of:

- Successful SSH access
- Linux command execution
- File and directory navigation
- File identification
- Searching for files
- Searching file contents
- Processing text
- Extracting strings
- Decoding Base64 data
- Password retrieval for progression between levels

The password outputs are visible in the practical screenshots and demonstrate successful completion of each corresponding challenge.

25. KEY LESSONS LEARNED

The Bandit practical provided several important lessons:

- Linux provides a powerful command-line environment.
- SSH can be used to access remote Linux systems.
- ls can be used to examine directory contents.
- cd can be used to navigate between directories.
- cat can be used to read files.
- Hidden files require appropriate command options to be displayed.
- Special filenames may require explicit path notation.
- Quotation marks can be used when filenames contain spaces.
- The file command can identify file types.
- Wildcards can simplify operations involving multiple files.
- find can locate files according to specific properties.
- Linux permissions can affect search results.
- 2>/dev/null can redirect unwanted error output.
- grep can efficiently search large text files.
- Command pipelines allow multiple commands to work together.
- sort and uniq can be used for data processing.
- strings can extract readable information from files.
- Base64 is encoding, not encryption.
- The man command provides useful documentation for Linux commands.
- Cybersecurity professionals need strong command-line skills.
- Problem-solving is as important as knowing individual commands.

26. CONCLUSION

Linux is an important foundation for cybersecurity because many security tools, servers, and security environments rely heavily on Linux and its command-line interface.

This task provided practical experience with Linux through the OverTheWire Bandit wargame. The practical focused on Levels 0-10, with each level introducing a different Linux command-line concept or problem-solving technique.

The practical began with connecting to the Bandit environment using SSH. The subsequent levels required navigating directories, reading files, handling special filenames, identifying hidden files, examining file types, searching for files based on properties, searching the filesystem, filtering text, identifying unique data, extracting readable strings, and decoding Base64 data.

The practical also demonstrated the importance of understanding how Linux interprets commands, filenames, paths, arguments, and output.

Rather than simply executing commands, each Bandit level required analyzing the problem and determining which Linux functionality could be used to solve it. This helped develop practical command-line problem-solving skills.

The skills developed during the practical are relevant to cybersecurity activities such as system administration, security analysis, digital forensics, incident response, penetration testing, and file investigation.

Overall, completing Bandit Levels 0-10 provided practical experience with Linux and strengthened my ability to work with the command line. The task also demonstrated that a strong understanding of Linux fundamentals is an important foundation for progressing into more advanced cybersecurity tools and techniques.